



摘要：

Anthropic 自家公司花在算力上的钱，也已经达到其薪资支出的 2.3 倍。按照一名高级工程师 22.4 万美元的完全成本来算，Anthropic 每位工程师每年对应的算力支出约为 51.5 万美元。也就是说：人还没模型贵。

“Fable 5 这个价格远高于中国程序员一天工资。写代码一天烧几百万 token 已经很节约了，然后一看账单几千 rmb。”

这是正在发生的现实。根据最新数据显示，Anthropic 自家公司花在算力上的钱，也已经达到其薪资支出的 2.3 倍。按照一名高级工程师 22.4 万美元的完全成本来算，Anthropic 每位工程师每年对应的算力支出约为 51.5 万美元。也就是说：人还没模型贵。

在这种账单面前，连 Claude 自己也不得不开始省 token 了。

Claude Code：烧 token 换“我很高产”的错觉

最近，业界又有了一个新词：Token Apocalypse（Token 末日）。

从 token maxing 到 token apocalypse，预示着 AI 行业真的发生了一种非常大的范式转变。今年三四月份，大家还在炫耀自己用了多少 token，甚至把它当成一种排行榜。但使用 AI 并不自动意味着省钱，于是大家开始更强调单个 token 的成本。

更微妙的是，大模型还在扩大很多原本根本不需要用 AI 的工作。我们现在 PDF 不想自己读了，长文不想自己看了，所有东西都要让 AI 总结。或者把这些东西用 AI 转成幻灯片，再丢给别人，对方可能再用 AI 读这些幻灯片……AI 像是在给一些本来就很虚的工作再强行注入一层价值，同时也把账单悄悄推高。

如今，成本失控已成常态。亚马逊、Adobe、Atlassian、花旗集团等公司开始对 AI 使用实施严格管控：

- 。限制模型等级：一些公司的员工被禁止使用 Claude Opus 等高端模型，被迫降级到更便宜的版本；
- 。设定个人限额：Uber 为每位工程师每月设定了 1500 美元的 token 上限；
- 。彻底停用权限：花旗银行等机构已完全限制对高级 AI 工具的访问，未达使用目标的员工甚至会被撤销企业账户。在此之前，Uber 的 CTO 曾坦言，公司在几个月内就用完了全年 AI 预算。Walmart 最近也停止了一些工具的使用。

大公司要么在四处找省钱办法，要么直接给 token 浪费踩急刹车。因此员工收到的信息极其矛盾：一边是“AI 能让你效率翻 100 倍，必须用”，一边是“别再把公司用破产了”。

这也是 AI 工具第一轮普及里最典型的问题：工具被推出时，并没有足够护栏来阻止公司在大语言模型上花掉数百万美元，也没有机制提醒团队 token 正在迅速烧光。不管是聊天机器人还是编码工具，很多产品先把“能用起来”摆在第一位，成本治理、使用配额、模型分级和上下文管理都被放到了后面。



但 Claude Code 本质上不是效率工具，而是一个营销工具。

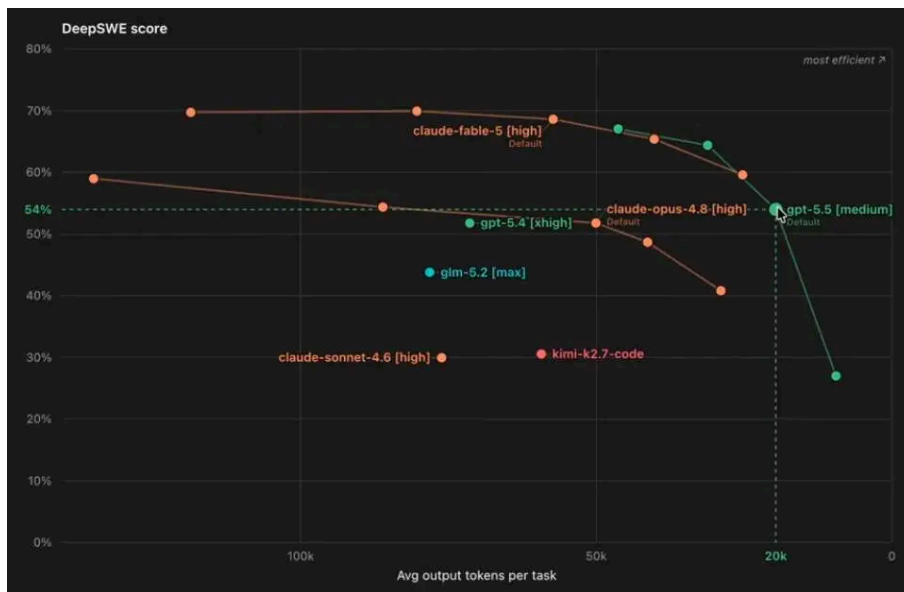
它的设计目标很明确：让你感觉自己在高产。Boris，Claude Code 的项目负责人，在做这个产品时最初的思考是：“如果模型变得足够聪明，代码会变成什么样？我希望如何使用这些东西？”——出发点不是“如何帮开发者省 token”，而是“如何展示模型的聪明”。

Anthropic 愿意为这种“感觉”烧掉大量 token——不管是你的钱，还是它们自己的钱。五分钟花掉 200 美元，对 Claude Code 来说不是事故，是设计。它的底层逻辑是：能多烧 token 解决的问题，绝不找更省 token 的办法。所有 sub-agent、所有花哨的 UI 动画、所有冗长的 reasoning trace，都不是为了效率，而是为了让你盯着屏幕时，觉得“这模型真聪明，真能干”。

这背后是一个精心设计的营销闭环：你烧掉大量 token，换来“高产”的感觉，于是觉得 Claude 好用，然后继续用它。Anthropic 甚至愿意自己承担大量 token 成本，来换取这种情绪上的认同。这也是为什么它们的桌面应用明显投入不足——Claude Code 的目标从来不是做一个好工具，而是成为 Anthropic 模型能力的“最佳展示窗口”。

而恰恰是这种“烧 token 换体验”的设计哲学，让 Claude 在 token 效率上被 OpenAI 甩开了。

OpenAI 一直在拼命压 token。从 reasoning trace 的压缩，到模型本身的效率优化，它们的哲学是：用更少的 token，干同样的活。Codex 5.5 就是最好的例子。



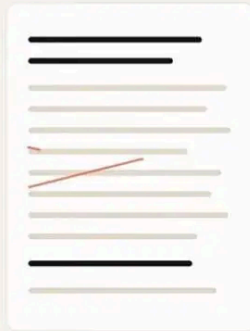
尽管像 Fable 5 这样的模型很智能，但与其他模型相比，它的效率不算高，Deep SWE 的这张图很能说明问题。如果将同批模型放一起对比，则更明显：GPT-5.5 medium 只用了 2 万个 token，就拿到了惊人的分数；而 Opus 4.8 用了 5 万个 token，得分反而更低。

这就是两条路线最直接的写照：行业在恐慌，Claude 在烧，OpenAI 在省。而接下来的问题就是——既然要降本，第一个该砍的是什么？答案是：那些堆了太久的提示词。

Claude Code 的 Prompt 债：堆得越多，欠得越多

在最新的演讲中，Anthropic 表示，他们已经删掉了 Claude Code 80% 的系统提示词。

We removed 80% of the system prompt from Claude Code



13 / 40

Anthropic 技术团队成员 Tariq Shihpar 解释说，这反映出 AI 模型引导方式正在发生一次根本变化——过去，人们认为指令越多、例子越多，模型表现就越好；但现在，这个逻辑不再成立。新模型 Fable 5 比它们自己给的示例更有想象力，示例反而成了限制。

这当然有营销成分，他吹嘘了一把 Fable 的能力：“示例反而容易限制模型，因为它实际上比我们给出的示例更有想象力”。但一个事实绕不过去：连 Anthropic 自己都开始对 system prompt 下刀了。



那么，为什么以前需要那么多 prompt？

过去一两年，AI Coding 圈形成了一套惯性思维：上下文越大越好，工具说明越多越好，system prompt 越完整越好。模型不知道项目怎么组织？写 Agents.md。模型不知道工具怎么用？写 tool descriptions。模型不够主动？写行为引导。模型不够稳定？继续往 system prompt 里加约束。

不可否认，system prompt 曾经是 AI Coding 工具的核心竞争力。对 LLM 的 prompt 做一些小调整，就可能带来显著的性能提升。如果同一个模型在 Codex、Cursor、OpenCode 和 Copilot 里的感觉不一样，那几乎肯定是因为 prompting 上存在细微差异。

这也是为什么 Cursor 曾花大量时间测试 system prompt，做 A/B testing，针对不同模型微调提示方式。与在 Claude Code 里使用 Opus 相比，Cursor 的 harness 能显著提升模型表现，一些 benchmark 测到的提升甚至高达 10% 到 30%。差别核心往往就是那几段 prompt。

但问题是只要 prompt 有用，团队就会不断往里加东西。某个模型喜欢乱用工具，就加一段规则；某个模型不够主动，就加一段鼓励；某个模型搜索太多，就补一段限制；某个模型不理解项目上下文，就再加一个 markdown 文件。每一次增加都有理由，但长期堆下来，system prompt 开始变成一个巨大的常驻上下文包袱。

问题在于：system prompt 不是免费的。它每次调用都要被读入、计费、占上下文。



Claude Code 把所有工具和功能内置进去之后，system prompt 一度膨胀到 65,000 个 token；即便关闭大部分功能，也还有 12,000 个 token。换句话说，模型还没开始写一行代码，就已经背上了一本说明书。对比来看，Pi 启动时上下文不到一千个 token。

更麻烦的是，prompt 债比代码债更隐蔽。

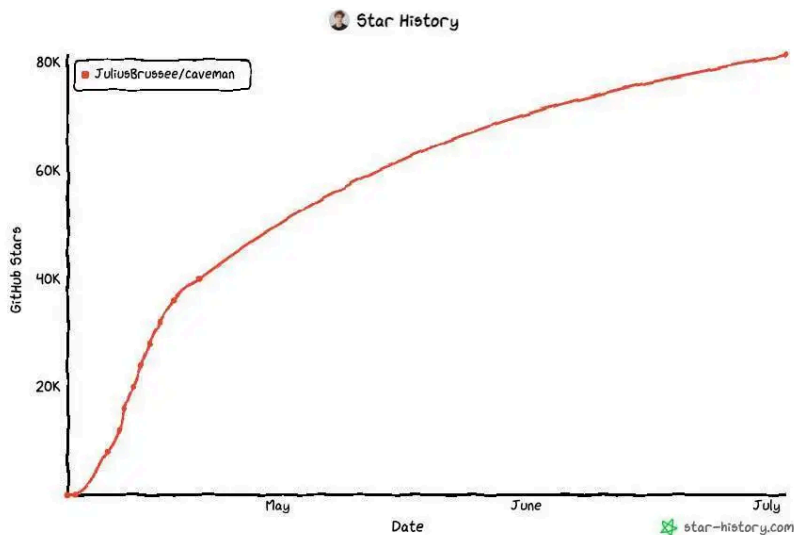
代码老了，通常会在改功能、跑测试、处理 bug 时暴露出来。Prompt 老了，却可能只是让模型悄悄变差。用户看到的是“Claude Code 最近好像不如以前聪明了”，或者“新模型没有宣传得那么强”，但真实原因可能是旧的 system prompt 没有跟上新模型。

当 prompt 从竞争力变成负担时，Anthropic 选择删掉 80%，也能进一步提升 token 效率。

Claude 的“废话税”：多说一个字，多花一份钱

Claude Code 的废话实在太多了。

今年有一个叫 Caveman 的插件迅速走红，专门解决这个问题。它的名字直译是“穴居人”，意思是像原始人一样说话——不讲礼貌，不加多余语法，不放填充词，只保留核心意思。



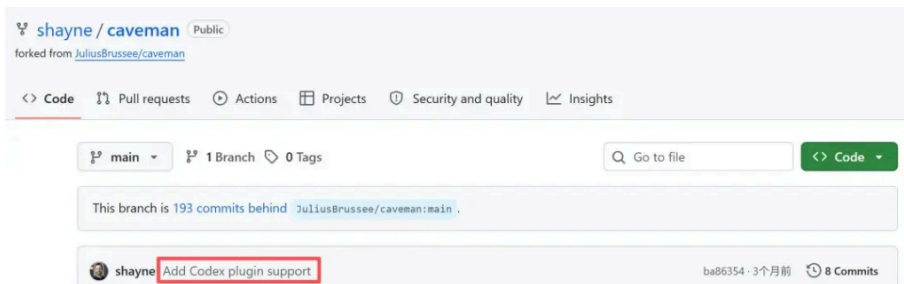
乍一看，它听起来像个玩笑。但一旦理解，你会发现它解决的是 LLM 里一个非常真实的问题：废话太多、token 太多、成本也不必要地变高。

而它的起源，正是针对 Claude Code。

“我是在 4 月初做出 Caveman 的，因为那段时间我重度使用 Claude Code，并且注意到我的很多 token 花费都浪费在了不必要的文字上：寒暄、模糊措辞、过渡语，以及那些在 agent loop 里其实并不重要的闲聊式表达。”Caveman 的创建者 Julius Brussee 说。

Brussee 的评测显示，Caveman 相比默认输出能减少 65% 到 75% 的输出 token，效果仍然超过普通的“请简洁”指令。它主要压缩的是周围的语言，不影响代码、命令、路径、URL、函数名这些需要精确性的部分。

据报道，OpenAI 的工程总监 Shayne Sweeney 也为该项目贡献了代码，以支持 Codex。



更有意思的是，OpenAI 早就把这种语言模式应用到了思考环节。

一些泄露出来的 reasoning trace （不是对外显示的 reasoning summary ）让外界看到了端倪。内容不像普通英语，更像压缩过的工程速记：

"Use core new nodes. Need infer. Need add VAE encode for images. Try. Try period."

这些句子看起来很好笑，甚至有点乱，但它们的重点不在可读性，而在 token 效率。模型在内部推理时，不需要像对用户说话那样保持礼貌、完整和流畅。它只需要保留动作、对象、判断和下一步。换句话说，只要最终答案是正常的，模型内部完全可以用一种更短、更粗糙、更省 token 的语言完成思考，以疯狂追求 token 效率。

这甚至比在写 Prompt 环节更有用。压缩 reasoning token 的收益更大，因为 agent 是多步执行的，前一步的思考会变成后一步的输入。模型每少“想”一段，省下来的就不只是当下这几个 token，而是后面整条执行链上的重复开销。

这正是 OpenAI 和 Claude 路线上的一个明显差异。

Claude 一直更好聊，也更像一个用完整语言思考和表达的助手。只要看看它的 reasoning trace 长很多，就能猜到它可能是在用普通英语。它的输出和 reasoning 往往更长，所以更依赖大上下文窗口来容纳这些内容。

这也是为什么 Claude 默认使用 100 万 token 的上下文窗口。很多人以为这是因为它想装进更大的代码库，但原因其实更简单：Claude 生成的东西太长了，没有这么大的窗口装不下。它们甚至在 compaction 上也很差，当你恢复旧线程时，Claude 会建议你不要保留完整上下文，而是尝试 compact。因为它们不会保留 reasoning trace——事实上，它们会在 10 到 20 分钟后清掉这些东西，因为 reasoning token 效率太低，不值得一直保留，否则成本会荒谬到不可接受。

而 OpenAI 模型的 token 上下文窗口大概是 20 万或更少，但因为它们一开始就通过这种简短语言做到了压缩。

一个值得品味的细节：如果 Anthropic 修复了“废话太多”这个问题，它们的收入会明显下降。如果开发者可以用模型完成同样的工作，但生成的 token 更少，那就是它们赚不到的钱。

来源：[InfoQ](#)

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

124 收藏

分享到:



写评论

请发表您的评论



表情



图片

发布评论

